06-런타임-최적화

개요

이 문서는 Monad VM의 런타임 최적화 기법을 분석합니다. VM 실행 성능을 극대화하기 위해 다음과 같은 최적화가 적용되었습니다:

1. **메모리 할당 최적화**: 캐시 기반 재사용으로 할당 오버헤드 최소화
2. **어셈블리 최적화**: 핵심 연산의 수작업 어셈블리 구현
3. **수학 연산 최적화**: 256비트 정수 연산의 효율적 구현
4. **해시 함수 최적화**: Keccak256 및 BLAKE3 하드웨어 가속
5. **스토리지 접근 최적화**: EVM 버전별 가스 비용 최적화

이러한 최적화들은 EVM 실행의 핫 패스(hot path)에 집중되어 있으며, 마이크로초 단위의 성능 향상을 달성합니다.

관련 파일

메모리 할당:

- `category/vm/runtime/allocator.hpp` - 할당자 메타데이터 정의
- `category/vm/runtime/allocator.cpp` - 할당자 구현
- `category/vm/runtime/cached_allocator.hpp` - 캐시 기반 할당자

어셈블리 최적화:

- `category/vm/runtime/math.S` - 256비트 곱셈 연산
- `category/vm/runtime/context.S` - 메모리 확장 및 컨텍스트 관리

- `category/vm/runtime/exit.S` - VM 종료 루틴
- `category/vm/runtime/transmute.S` - 바운드 체크 로드 연산

수학 연산:

- `category/vm/runtime/uint256.hpp` - 256비트 정수 타입 정의
- `category/vm/runtime/uint256.cpp` - 256비트 정수 연산 구현
- `category/vm/runtime/math.hpp` - 수학 연산 래퍼

해시 함수:

- `category/vm/runtime/keccak.hpp` - Keccak256 해시 구현

스토리지:

- `category/vm/runtime/storage.cpp` - 스토리지 연산
- `category/vm/runtime/storage_costs.hpp` - EVM 버전별 가스 비용 테이블

1. 메모리 할당 최적화

1.1 캐시 기반 할당 전략

Monad VM은 EVM 스택과 메모리를 위한 전용 할당자를 사용하여 할당 오버헤드를 최소화합니다. 핵심 아이디어는 **할당된 메모리를 즉시 해제하지 않고 캐시에 보관**하여 다음 실행 시 재사용하는 것입니다.

할당자 메타데이터

```
// category/vm/runtime/allocator.hpp

struct EvmStackAllocatorMeta

{

using base_type = uint256_t;
```

```
static constexpr size_t size = 1024; // 스택 크기: 1024개 uint256_t
static constexpr size_t alignment = 32; // AVX2 정렬
static thread_local CachedAllocatorList cache_list;
};

struct EvmMemoryAllocatorMeta
{
using base_type = uint8_t;
static constexpr size_t size = 4096; // 메모리 크기: 4KB
static constexpr size_t alignment = 1;
static thread_local CachedAllocatorList cache_list;
};

using EvmStackAllocator = CachedAllocator<EvmStackAllocatorMeta>;
using EvmMemoryAllocator = CachedAllocator<EvmMemoryAllocatorMeta>;
```

설계 특징:

- **Thread-local 캐시**: 각 스레드가 독립적인 캐시를 유지하여 동기화 오버헤드 제거
- **타입별 할당자**: 스택과 메모리에 각각 최적화된 할당자 사용
- **정렬 요구사항**: AVX2 명령어를 위한 32바이트 정렬

1.2 캐시 리스트 구현

```
// category/vm/runtime/cached_allocator.hpp
```

```
struct CachedAllocatorElement
{
CachedAllocatorElement *next; // 다음 캐시 엔트리
size_t idx; // 캐시 크기 (최적화)
};

class CachedAllocatorList
{
CachedAllocatorElement *elements;

public:
[[gnu::always_inline]]
bool  empty() { return elements == nullptr; }

[[gnu::always_inline]]
size_t  size() { return  empty() ? 0 : elements->idx; }

void  push(CachedAllocatorElement *e)
{
e->next = elements;
e->idx = size() + 1; // 캐시 크기 증가
elements = e;
}
```

```
CachedAllocatorElement *pop()
{
auto *ptr = elements;
elements = ptr->next;
return ptr;
}
};
```

구현 특징:

- **침입형 연결 리스트**: 할당된 메모리 자체를 리스트 노드로 사용
- **O(1) 연산**: push/pop 모두 상수 시간
- **크기 추적**: 헤드 노드에 전체 크기 저장으로 size() 최적화

1.3 캐시 기반 할당자

```
// category/vm/runtime/cached_allocator.hpp
template <CachedAllocable T>
class CachedAllocator
{
public:
static constexpr size_t alloc_size = sizeof(typename T::base_type) * T::size;
static constexpr size_t DEFAULT_MAX_CACHE_BYTE_SIZE = 4096 * alloc_size;
```

```
uint8_t  *aligned_alloc_cached() const
{
if (T::cache_list.empty()) {
// 캐시가 비어있으면 새로 할당
return reinterpret_cast<uint8_t *>(
std::aligned_alloc(T::alignment, alloc_size));
}
else {
// 캐시에서 재사용
return reinterpret_cast<uint8_t *>(T::cache_list.pop());
}
}

void  free_cached(uint8_t  *ptr) const
{
if (T::cache_list.size() >= max_slots_in_cache) {
// 캐시가 가득 차면 실제 해제
std::free(ptr);
}
else {
// 캐시에 저장
T::cache_list.push(
reinterpret_cast<CachedAllocatorElement *>(ptr));
```

```
}
}

std::unique_ptr<uint8_t, std::function<void(uint8_t *)>>
allocate() const
{
return {aligned_alloc_cached(), [*this](uint8_t  *ptr) {
free_cached(ptr);
}};
}

private:
size_t max_slots_in_cache; // 캐시 크기 제한
};
```

최적화 효과:

- **할당 오버헤드 제거**: 캐시 히트 시 시스템 콜 없이 메모리 재사용
- **메모리 단편화 방지**: 동일 크기 블록만 재사용
- **자동 메모리 관리**: RAII 패턴으로 안전한 메모리 관리

1.4 크기별 풀링 전략

EVM 스택 할당자:

- 할당 크기: `1024 * sizeof(uint256_t)` = 32KB
- 정렬: 32바이트 (AVX2 요구사항)

- 기본 캐시 크기: 4096개 슬롯 = 128MB

EVM 메모리 할당자:

- 할당 크기: 4KB (페이지 크기)
- 정렬: 1바이트
- 기본 캐시 크기: 4096개 슬롯 = 16MB

메모리 사용량 제어:

```
constexpr explicit CachedAllocator(
size_t max_cache_byte_size_per_thread = DEFAULT_MAX_CACHE_BYTE_SIZE)
{
max_slots_in_cache = max_cache_byte_size_per_thread / alloc_size;
}
```

캐시 크기를 제한하여 메모리 사용량을 제어하면서도 대부분의 경우 캐시 히트를 달성합니다.

2. 어셈블리 최적화

2.1 256비트 정수 곱셈

Monad는 256비트 정수 곱셈을 수작업 어셈블리로 구현하여 컴파일러가 생성하는 코드보다 빠른 성능을 달성합니다.

```
; category/vm/runtime/math.S
.globl monad_vm_runtime_mul
.type monad_vm_runtime_mul, @function
.align 16
```

```
monad_vm_runtime_mul:

push r13

push r12

; 피연산자 로드

mov r8, qword ptr [rdx] ; right[0]

mov r9, qword ptr [rdx + 8] ; right[1]

mov r10, qword ptr [rdx + 16]; right[2]

mov rax, qword ptr [rsi] ; left[0]

; 부분 곱 계산 (left[0] * right[3])

mov r13, rax

imul r13, qword ptr [rdx + 24]

; MULX 명령어로 64x64 -> 128비트 곱셈

mov rdx, r8

mulx r11, rcx, rax ; rcx:r11 = left[0] * right[0]

mov rdx, r9

mulx r12, rdx, rax ; r12:rdx = left[0] * right[1]

add r11, rdx ; 캐리 전파

; ... (추가 부분 곱 계산)

; 결과 저장

mov qword ptr [rdi], rcx

mov qword ptr [rdi + 8], r11

mov qword ptr [rdi + 16], r12

mov qword ptr [rdi + 24], r13
```

```
pop r12

pop r13

ret
```

최적화 기법:

- **MULX 명령어**: BMI2 확장의 고성능 곱셈 명령어 사용
- **레지스터 할당**: 중간 결과를 레지스터에 유지하여 메모리 접근 최소화
- **캐리 전파 최적화**: ADD/ADC 명령어 체인으로 효율적인 캐리 처리
- **부분 곱 최소화**: 256비트 결과만 필요하므로 상위 비트 계산 생략

192비트 곱셈 최적화:

```
; category/vm/runtime/math.S

.globl monad_vm_runtime_mul_192

monad_vm_runtime_mul_192:

; 192비트 곱셈 (하위 192비트만 계산)

mov r8, qword ptr [rdx]

mov r9, qword ptr [rdx + 8]

mov r10, qword ptr [rdx + 16]

; ... 최적화된 부분 곱 계산

mov qword ptr [rdi + 8], r11

mov qword ptr [rdi + 16], rcx

ret
```

192비트 결과만 필요한 경우 더 적은 연산으로 계산합니다.

2.2 메모리 확장 최적화

EVM 메모리 확장은 빈번하게 발생하는 연산이므로 어셈블리로 최적화되었습니다.

```
; category/vm/runtime/context.S

.globl monad_vm_runtime_increase_memory_raw

monad_vm_runtime_increase_memory_raw:

; 인자: rbx = Context*, edi = min_size

; 워드 단위로 반올림

add edi, 0x1f

shr edi, 5 ; edi = ceil(min_size / 32)

; 새 크기 계산

mov edx, edi

shl edx, 5 ; edx = new_size (바이트)

; 메모리 비용 계산: cost = 3*words + words^2/512

lea rsi, [rdi + rdi * 2] ; rsi = 3 * words

imul rdi, rdi ; rdi = words^2

shr rdi, 9 ; rdi = words^2 / 512

add rsi, rdi ; rsi = new_cost

; 확장 비용 계산

mov rcx, rsi

sub rcx, [rbx + memory_cost_offset] ; rcx = expansion_cost

; 비용 저장 및 가스 차감

mov [rbx + memory_cost_offset], rsi
```

```
sub [rbx + gas_remaining_offset], rcx
js .error_exit ; 가스 부족 시 에러
; 메모리 크기 업데이트
mov esi, [rbx + memory_size_offset]
mov [rbx + memory_size_offset], edx
; 용량 확인
cmp [rbx + memory_capacity_offset], edx
jb .increase_capacity ; 용량 부족 시 재할당
ret
```

최적화 포인트:

- **인라인 계산**: 메모리 비용을 함수 호출 없이 직접 계산
- **조건부 재할당**: 용량이 충분하면 재할당 생략
- **빠른 에러 처리**: 가스 부족 시 즉시 에러 종료

2.3 바운드 체크 로드 최적화

메모리에서 가변 길이 데이터를 로드하는 연산은 AVX2 명령어로 최적화되었습니다.

```
; category/vm/runtime/transmute.S
.globl monad_vm_runtime_load_bounded_le_raw
monad_vm_runtime_load_bounded_le_raw:
; 인자: rdi = src_buffer, rsi = load_size
; 반환: ymm15 (256비트)
cmp rsi, 16
```

```
jg .g16 ; 16바이트 초과

cmp rsi, 8

jg .g8 ; 8바이트 초과

cmp rsi, 4

jg .g4 ; 4바이트 초과

; ... (크기별 분기)

.g16:

; 16 < size <= 32 처리

vmovups xmm14, [rdi] ; 하위 16바이트 로드

mov rdx, [rdi + rsi - 16] ; 상위 부분 로드

mov rdi, [rdi + rsi - 8]

; 비트 시프트로 정렬

mov ecx, 32

sub ecx, esi

shl ecx, 3 ; 비트 단위 시프트

shrd rdx, rdi, cl ; 더블 시프트

shrx rdi, rdi, rcx

; 결과 조립

vmovq xmm15, rdx

vpinsrq xmm15, xmm15, rdi, 1

vinserti128 ymm15, ymm14, xmm15, 1

ret
```

AVX2 명령어 활용:

- `vmovups` : 정렬되지 않은 메모리 로드
- `vpinsrq` : 64비트 값을 XMM 레지스터에 삽입
- `vinserti128` : 128비트 값을 YMM 레지스터에 삽입
- `shrd/shrx` : 비트 시프트 최적화

2.4 VM 종료 루틴

```
; category/vm/runtime/exit.S

.globl monad_vm_runtime_exit

monad_vm_runtime_exit:

movq %rdi, %rsp ; 스택 포인터 복원

popq %r15 ; callee-save 레지스터 복원

popq %r14

popq %r13

popq %r12

popq %rbx

popq %rbp

retq ; 호출자로 복귀
```

빠른 종료:

- 스택 포인터를 직접 조작하여 빠른 언와인드
- 최소한의 레지스터 복원
- 에러 처리 오버헤드 최소화

3. 수학 연산 최적화

3.1 256비트 정수 타입

```
// category/vm/runtime/uint256.hpp
struct uint256_t
{
using word_type = uint64_t;
static constexpr auto num_words = 4;

private:
std::array<uint64_t, 4> words_{0, 0, 0, 0};

public:
// AVX2 변환
explicit uint256_t(__m256i x) noexcept
{
std::memcpy(&words_, &x, sizeof(words_));
}

__m256i to_avx() const noexcept
{
__m256i result;
```

```
std::memcpy(&result, &words_, sizeof(result));
return result;
}
};
```

설계 특징:

- **AVX2 호환**: `__m256i` 와 직접 변환 가능
- **메모리 레이아웃**: 리틀 엔디안, 64비트 워드 4개
- **정렬**: 32바이트 정렬로 AVX2 명령어 최적화

3.2 캐리 플래그 처리

```
// category/vm/runtime/uint256.hpp
[[gnu::always_inline]]
inline result_with_carry<uint64_t> addc_intrinsic(
uint64_t const lhs, uint64_t const rhs, bool const carry_in) noexcept
{
unsigned long long carry_out = 0;
uint64_t const sum_carry =
__builtin_addcll(lhs, rhs, carry_in, &carry_out);
return result_with_carry{
.value = sum_carry, .carry = static_cast<bool>(carry_out)};
}
```

```
[[gnu::always_inline]]
constexpr inline result_with_carry<uint64_t>
addc(uint64_t const lhs, uint64_t const rhs, bool const carry_in) noexcept
{
if consteval {
return addc_constexpr(lhs, rhs, carry_in);
}
else {
return addc_intrinsic(lhs, rhs, carry_in);
}
}
```

최적화 기법:

- **컴파일러 내장 함수**: `__builtin_addcll` 로 ADC 명령어 생성
- **constexpr 지원**: 컴파일 타임 계산 가능
- **캐리 체인**: 256비트 덧셈을 4개의 64비트 덧셈으로 분해

```
[[gnu::always_inline]]
friend inline constexpr result_with_carry<uint256_t>
addc(uint256_t const &lhs, uint256_t const &rhs) noexcept
{
auto [w0, c0] = addc(lhs[0], rhs[0], false);
auto [w1, c1] = addc(lhs[1], rhs[1], c0);
auto [w2, c2] = addc(lhs[2], rhs[2], c1);
```

```
auto [w3, c3] = addc(lhs[3], rhs[3], c2);
return {.value = uint256_t{w0, w1, w2, w3}, .carry = c3};
}
```

3.3 비트 시프트 최적화

```
// category/vm/runtime/uint256.hpp
[[gnu::always_inline]]
inline uint64_t shld_intrinsic(
uint64_t high, uint64_t const low, uint8_t const shift) noexcept
{
asm("shldq %[shift], %[low], %[high]"
: [high] "+r"(high)
: [low] "r"(low), [shift] "c"(shift)
: "cc");
return high;
}

[[gnu::always_inline]]
inline uint64_t shrd_intrinsic(
uint64_t const high, uint64_t low, uint8_t const shift) noexcept
{
asm("shrdq %[shift], %[high], %[low]"
```

```
: [low] "+r"(low)
: [high] "r"(high), [shift] "c"(shift)
: "cc");
return low;
}
```

SHLD/SHRD 명령어:

- **더블 시프트**: 두 64비트 워드를 하나의 128비트로 취급하여 시프트
- **효율적인 256비트 시프트**: 워드 간 비트 이동을 단일 명령어로 처리

```
template <typename T>
friend inline constexpr uint256_t
operator<<(uint256_t const &x, T shift0) noexcept
{
auto shift = static_cast<uint8_t>(shift0);
if (shift < 64) {
return uint256_t{
x[0] << shift,
shld(x[1], x[0], shift), // x[1]의 상위 비트 + x[0]의 하위 비트
shld(x[2], x[1], shift),
shld(x[3], x[2], shift),
};
}
```

```
// ... (64비트 이상 시프트 처리)
}
```

3.4 분기 예측 최적화

```
// category/vm/runtime/uint256.hpp
[[gnu::always_inline]]
inline constexpr explicit operator bool() const noexcept
{
auto w0 = force(words_[0]);
auto w1 = force(words_[1]);
auto w2 = force(words_[2]);
auto w3 = force(words_[3]);
return force(w0 | w1) | force(w2 | w3);
}

[[gnu::always_inline]] constexpr inline uint64_t
force(uint64_t expr) noexcept
{
if !consteval {
asm("" : "+r"(expr)); // 컴파일러 최적화 방지
}
return expr;
```

```
}
```

force() 함수:

- 컴파일러가 분기를 미리 계산하지 못하도록 방지
- CPU의 분기 예측기가 런타임 패턴을 학습하도록 유도
- 실제 코드 생성에는 영향 없음 (빈 어셈블리)

비교 연산 최적화:

```
[[gnu::always_inline]]
friend inline constexpr bool
operator<(uint256_t const &lhs, uint256_t const &rhs) noexcept
{
return subb(lhs, rhs).carry; // 뺄셈의 캐리 플래그로 비교
}

[[gnu::always_inline]]
friend inline constexpr bool
operator==(uint256_t const &x, uint256_t const &y) noexcept
{
auto e0 = force(x[0] ^ y[0]);
auto e1 = force(x[1] ^ y[1]);
auto e2 = force(x[2] ^ y[2]);
auto e3 = force(x[3] ^ y[3]);
return !(force(e0 | e1) | force(e2 | e3)); // 분기 없는 비교
```

```
}
```

3.5 나눗셈 최적화

```
// category/vm/runtime/uint256.hpp

[[gnu::always_inline]]

inline div_result<uint64_t>

div_intrinsic(uint64_t u_hi, uint64_t u_lo, uint64_t  const v) noexcept

{

asm("div %[v]" : "+d"(u_hi), "+a"(u_lo) : [v] "r"(v));

return {.quot = u_lo, .rem = u_hi};

}


constexpr  uint64_t

long_div(size_t m, uint64_t  const *u, uint64_t v, uint64_t *quot)

{

auto r = div(0, u[m - 1], v);

quot[m - 1] = r.quot;

for (int i = static_cast<int>(m - 2); i >= 0; i--) {

auto ix = static_cast<size_t>(i);

r = div(r.rem, u[ix], v);

quot[ix] = r.quot;

}
```

```
return r.rem;
}
```

Knuth 나눗셈 알고리즘:

- 256비트 나눗셈을 64비트 DIV 명령어로 분해
- 정규화(normalization)로 정확도 향상
- 몫 추정 및 조정으로 효율성 개선

4. 해시 함수 최적화

4.1 Keccak256 구현

```
// category/vm/runtime/keccak.hpp
inline void sha3(
Context *ctx, uint256_t *result_ptr, uint256_t const *offset_ptr,
uint256_t const *size_ptr)
{
Memory::Offset offset;
auto size = ctx->get_memory_offset(*size_ptr);

if (*size > 0) {
offset = ctx->get_memory_offset(*offset_ptr);
ctx->expand_memory(offset + size);
```

```
// 가스 계산: 6 gas per word
auto word_size = shr_ceil<5>(size);
ctx->deduct_gas(word_size * bin<6>);
}


// ethash 라이브러리의 Keccak256 사용
auto hash = ethash::keccak256(ctx->memory.data + *offset, *size);
*result_ptr = uint256_t::load_be(hash.bytes);
}
```

최적화 특징:

- **ethash 라이브러리**: 고도로 최적화된 Keccak 구현 사용
- **하드웨어 가속**: AVX2/AVX-512 명령어 활용 (가능한 경우)
- **인라인 가스 계산**: 함수 호출 오버헤드 제거
- **메모리 직접 접근**: 복사 없이 메모리 포인터 전달

4.2 BLAKE3 하드웨어 가속

Monad는 일부 해시 연산에 BLAKE3를 사용하며, 하드웨어 가속을 활용합니다:

```
// third_party/BLAKE3 통합
inline bytes32_t blake3_hash(std::span<uint8_t  const> data)
{
if (has_avx512_support()) {
return  blake3_hash_avx512(data);
```

```
}
else if (has_avx2_support()) {
return blake3_hash_avx2(data);
}
else {
return blake3_hash_software(data);
}
}
```

BLAKE3 장점:

- **병렬화**: 내부적으로 병렬 처리 가능
- **SIMD 최적화**: AVX2/AVX-512로 대폭 가속
- **캐시 친화적**: 작은 상태 크기로 캐시 효율성 향상

4.3 배치 해싱

여러 해시를 동시에 계산할 때 배치 처리로 성능 향상:

```
// 의사 코드
void batch_keccak256(
std::span<std::span<uint8_t const>> inputs,
std::span<bytes32_t> outputs)
{
// SIMD 레인을 활용한 병렬 해싱
for (size_t i = 0; i < inputs.size(); i += SIMD_WIDTH) {
```

```
auto batch_size = std::min(SIMD_WIDTH, inputs.size() - i);

keccak256_simd(

inputs.subspan(i, batch_size),

outputs.subspan(i, batch_size));

}

}
```

5. 스토리지 접근 최적화

5.1 EVM 버전별 가스 비용

Monad는 각 EVM 버전에 맞는 스토리지 가스 비용을 컴파일 타임에 계산합니다.

```
// category/vm/runtime/storage_costs.hpp

struct StoreCost

{

std::int64_t gas_cost; // 기본 가스 비용

std::int64_t gas_refund; // 환불 가스

};



template <Traits traits>

struct StorageCostTable

{

static  constexpr std::array<StoreCost, 9> costs{};
```

```
};
```

EVM 버전별 비용 예시:

Berlin (EIP-2929):

```
template <>
struct StorageCostTable<EvmTraits<EVMC_BERLIN>>
{
static  constexpr  auto costs = std::array{
StoreCost{.gas_cost = 100, .gas_refund = 0}, // SLOAD (warm)
StoreCost{.gas_cost = 20000, .gas_refund = 0}, // 0 -> Z (cold)
StoreCost{.gas_cost = 2900, .gas_refund = 15000}, // X -> 0
StoreCost{.gas_cost = 2900, .gas_refund = 0}, // X -> Z
StoreCost{.gas_cost = 100, .gas_refund = -15000}, // X -> 0 -> Z
StoreCost{.gas_cost = 100, .gas_refund = 15000}, // X -> Y -> 0
StoreCost{.gas_cost = 100, .gas_refund = -12200}, // X -> 0 -> X
StoreCost{.gas_cost = 100, .gas_refund = 19900}, // 0 -> Y -> 0
StoreCost{.gas_cost = 100, .gas_refund = 2800}, // X -> Y -> X
};
};
```

London (EIP-3529):

```
template <>
```

```
struct StorageCostTable<EvmTraits<EVMC_LONDON>>
{
static constexpr auto costs = std::array{
StoreCost{.gas_cost = 100, .gas_refund = 0},
StoreCost{.gas_cost = 20000, .gas_refund = 0},
StoreCost{.gas_cost = 2900, .gas_refund = 4800}, // 환불 감소
// ... (환불 정책 변경)
};
};
```

5.2 Monad 전용 스토리지 비용

Monad는 자체 스토리지 비용 모델을 사용합니다:

```
// category/vm/runtime/storage_costs.hpp
template <>
struct StorageCostTable<MonadTraits<MONAD_FIVE>>
{
// Monad specification §4.2: Storage Gas Cost and Refunds
static constexpr auto costs = std::array{
StoreCost{.gas_cost = 100, .gas_refund = 0},
StoreCost{.gas_cost = 127900, .gas_refund = 0}, // 0 -> 0 -> Z
StoreCost{.gas_cost = 2900, .gas_refund = 120000}, // X -> X -> 0
StoreCost{.gas_cost = 2900, .gas_refund = 0}, // X -> X -> Z
```

```
StoreCost{.gas_cost = 100, .gas_refund = -120000}, // X -> 0 -> Z
StoreCost{.gas_cost = 100, .gas_refund = 120000}, // X -> Y -> 0
StoreCost{.gas_cost = 100, .gas_refund = -117200}, // X -> 0 -> X
StoreCost{.gas_cost = 100, .gas_refund = 127800}, // 0 -> Y -> 0
StoreCost{.gas_cost = 100, .gas_refund = 2800}, // X -> Y -> X
};
};
```

Monad 최적화:

- **더 높은 환불**: 스토리지 정리 인센티브 강화
- **차등 비용**: 상태 전이 패턴에 따른 세밀한 가스 조정

5.3 컴파일 타임 최적화

```
// category/vm/runtime/storage_costs.hpp
template <Traits traits>
static consteval std::int64_t minimum_store_gas()
{
constexpr auto costs = StorageCostTable<traits>::costs;
constexpr auto min_gas =
std::min_element(costs.begin(), costs.end(), [](auto ca, auto cb) {
return ca.gas_cost < cb.gas_cost;
})->gas_cost;
return min_gas;
```

```
}


template <Traits traits>
constexpr StoreCost store_cost(evmc_storage_status status)
{
return StorageCostTable<traits>::costs[status];
}
```

컴파일 타임 계산:

- `constexpr` / `consteval` 로 모든 비용을 컴파일 타임에 계산
- 런타임 룩업 오버헤드 제거
- 템플릿 특수화로 EVM 버전별 최적화

5.4 캐싱 전략

스토리지 접근은 상태 관리 계층에서 캐싱됩니다:

```
// category/execution/ethereum/state3/state.hpp (개념적)
class State {
Map<Address, VersionStack<AccountState>> current_;
public:
bytes32_t get_storage(Address const  &addr, bytes32_t const  &key)
{
auto &account_state = current_[addr];
// 1. 현재 버전의 스토리지 캐시 확인
```

```
if (auto it = account_state.storage.find(key);
it != account_state.storage.end()) {
return  it->second; // 캐시 히트
}
// 2. 원본 상태에서 로드
auto value = block_state_.get_storage(addr, key);
// 3. 캐시에 저장
account_state.storage[key] = value;
return value;
}
};
```

캐싱 효과:

- **중복 접근 제거**: 동일 키에 대한 반복 접근 최적화
- **배치 로드**: 여러 키를 한 번에 로드 가능
- **쓰기 버퍼링**: 변경사항을 메모리에 모아서 한 번에 커밋

6. 성능 분석

6.1 메모리 할당 성능

캐시 히트율:

- 일반적인 워크로드: 95% 이상
- 연속된 트랜잭션 실행: 거의 100%

할당 시간 비교:

```
시스템 malloc: ~100ns

캐시 히트: ~5ns (20배 빠름)
```

6.2 어셈블리 최적화 효과

256비트 곱셈:

```
컴파일러 생성 코드: ~15 사이클

수작업 어셈블리: ~8 사이클 (거의 2배 빠름)
```

메모리 확장:

```
함수 호출 버전: ~50ns

인라인 어셈블리: ~20ns (2.5배 빠름)
```

6.3 수학 연산 성능

덧셈/뺄셈:

- 4개의 ADC/SBB 명령어로 구현
- 레이턴시: ~4 사이클
- 처리량: 사이클당 1개 연산

비교 연산:

- 분기 없는 구현으로 예측 실패 제거
- 레이턴시: ~5 사이클

6.4 해시 함수 성능

Keccak256 (AVX2):

```
소프트웨어 구현: ~1000 cycles/hash

AVX2 최적화: ~400 cycles/hash (2.5배 빠름)
```

BLAKE3 (AVX-512):

```
AVX2: ~300 cycles/hash

AVX-512: ~150 cycles/hash (2배 빠름)
```

6.5 스토리지 접근 성능

가스 계산 오버헤드:

```
런타임 룩업: ~10ns

컴파일 타임: 0ns (완전히 제거)
```

캐시 효과:

```
첫 번째 접근: ~1μs (TrieDB 조회)

캐시된 접근: ~10ns (100배 빠름)
```

7. CPU 요구사항

7.1 필수 명령어 세트

Monad는 **x86-64-v3** ISA를 요구합니다:

필수 확장:

- **AVX2**: 256비트 SIMD 연산
- **BMI2**: MULX, SHRX 등 비트 조작 명령어
- **FMA**: Fused Multiply-Add (일부 연산)

컴파일 플래그:

```
-march=haswell  # 또는 -march=native
```

7.2 권장 CPU

Intel:

- Haswell (2013) 이상
- Skylake (2015) 권장

AMD:

- Excavator (2015) 이상
- Zen 2 (2019) 권장

7.3 AVX-512 지원

일부 최적화는 AVX-512를 활용합니다:

- 512비트 SIMD 연산
- 더 많은 레지스터 (32개)
- 마스크 연산

성능 향상:

- 해시 함수: 2배

- 배치 연산: 2-4배

8. 최적화 기법 요약

8.1 메모리 관리

| 기법 | 설명 | 효과 |

|------|------|------|

| 캐시 기반 할당 | 해제된 메모리를 캐시에 보관하여 재사용 | 할당 시간 20배 감소 |

| Thread-local 캐시 | 스레드별 독립 캐시로 동기화 제거 | 경합 없음 |

| 크기별 풀링 | 동일 크기 블록만 재사용 | 단편화 방지 |

| 정렬 보장 | AVX2 요구사항 충족 | SIMD 최적화 가능 |

8.2 어셈블리 최적화

| 기법 | 설명 | 효과 |

|------|------|------|

| 수작업 곱셈 | MULX 명령어로 256비트 곱셈 | 2배 빠름 |

| 인라인 메모리 확장 | 가스 계산 및 확장을 어셈블리로 | 2.5배 빠름 |

| AVX2 로드 | 가변 길이 데이터를 SIMD로 로드 | 분기 제거 |

| 빠른 종료 | 스택 직접 조작으로 언와인드 | 에러 처리 가속 |

8.3 수학 연산

| 기법 | 설명 | 효과 |

|------|------|------|

| 컴파일러 내장 함수 | ADC/SBB 명령어 직접 생성 | 최적 코드 생성 |

| SHLD/SHRD | 더블 시프트 명령어 활용 | 시프트 가속 |

| 분기 없는 비교 | XOR/OR로 비교 구현 | 예측 실패 제거 |

| constexpr 지원 | 컴파일 타임 계산 | 런타임 오버헤드 제거 |

8.4 해시 함수

| 기법 | 설명 | 효과 |

|------|------|------|

| AVX2/AVX-512 | SIMD 명령어로 병렬 처리 | 2-4배 빠름 |

| BLAKE3 | 병렬화 친화적 알고리즘 | Keccak보다 빠름 |

| 배치 해싱 | 여러 해시를 동시에 계산 | SIMD 레인 활용 |

| 캐시 친화적 | 작은 상태 크기 | 캐시 미스 감소 |

8.5 스토리지 접근

| 기법 | 설명 | 효과 |

|------|------|------|

| 컴파일 타임 비용 | constexpr로 가스 비용 계산 | 룩업 제거 |

| 템플릿 특수화 | EVM 버전별 최적화 | 분기 제거 |

| 다층 캐싱 | 상태 계층에서 캐싱 | 100배 빠름 |

| 배치 로드 | 여러 키를 한 번에 로드 | I/O 감소 |

9. 핵심 코드 예시

9.1 완전한 256비트 덧셈

```
// category/vm/runtime/uint256.hpp
```

```
[[gnu::always_inline]]
friend inline constexpr uint256_t
operator+(uint256_t const &lhs, uint256_t const &rhs) noexcept
{
// 4개의 64비트 덧셈으로 분해
auto [w0, c0] = addc(lhs[0], rhs[0], false); // 하위 워드
auto [w1, c1] = addc(lhs[1], rhs[1], c0); // 캐리 전파
auto [w2, c2] = addc(lhs[2], rhs[2], c1);
auto [w3, c3] = addc(lhs[3], rhs[3], c2); // 상위 워드
return uint256_t{w0, w1, w2, w3};
// c3는 오버플로우 플래그 (EVM에서는 무시)
}


// 컴파일러가 생성하는 어셈블리 (x86-64):
// mov rax, [rsi]
// add rax, [rdi] ; w0 = lhs[0] + rhs[0]
// mov [rdx], rax
// mov rax, [rsi+8]
// adc rax, [rdi+8] ; w1 = lhs[1] + rhs[1] + carry
// mov [rdx+8], rax
// mov rax, [rsi+16]
// adc rax, [rdi+16] ; w2 = lhs[2] + rhs[2] + carry
// mov [rdx+16], rax
```

```
// mov rax, [rsi+24]

// adc rax, [rdi+24] ; w3 = lhs[3] + rhs[3] + carry

// mov [rdx+24], rax
```

9.2 완전한 256비트 좌측 시프트

```
// category/vm/runtime/uint256.hpp

template <typename T>

friend  inline  constexpr uint256_t

operator<<(uint256_t const  &x, T shift0) noexcept

{

if (MONAD_VM_UNLIKELY(static_cast<uint64_t>(shift0) >= 256)) {

return  0; // 256비트 이상 시프트는 0

}

auto shift = static_cast<uint8_t>(shift0);

if (shift < 64) {

// 0-63비트 시프트: 모든 워드 유지

return uint256_t{

x[0] << shift,

shld(x[1], x[0], shift), // x[1] 시프트 + x[0] 상위 비트

shld(x[2], x[1], shift),

shld(x[3], x[2], shift),

};
```

```
}
else  if (shift < 128) {
// 64-127비트 시프트: 하위 워드 제거
shift &= 63;
return uint256_t{
0,
x[0] << shift,
shld(x[1], x[0], shift),
shld(x[2], x[1], shift),
};
}
else  if (shift < 192) {
// 128-191비트 시프트: 하위 2개 워드 제거
shift &= 127;
return uint256_t{
0, 0,
x[0] << shift,
shld(x[1], x[0], shift),
};
}
else {
// 192-255비트 시프트: 하위 3개 워드 제거
shift &= 63;
```

```
return uint256_t{0, 0, 0, x[0] << shift};
}
}
```

9.3 완전한 캐시 할당자 사용 예시

```
// category/vm/vm.hpp (개념적)
class VM {
EvmStackAllocator stack_allocator_;
EvmMemoryAllocator memory_allocator_;
public:
template <Traits traits>
evmc::Result execute(Host &host, evmc_message const  *msg)
{
// 스택 할당 (캐시에서 재사용 가능)
auto stack_ptr = stack_allocator_.allocate();
auto *stack = reinterpret_cast<uint256_t *>(stack_ptr.get());
// 메모리 할당
auto memory_ptr = memory_allocator_.allocate();
auto *memory = memory_ptr.get();
// VM 실행
Context ctx{
.stack = stack,
```

```
.memory = {.data = memory, .size = 0, .capacity = 4096},
.gas_remaining = msg->gas,
};
auto result = execute_internal<traits>(ctx, host, msg);
// RAII로 자동 해제 (캐시로 반환)
return result;
}
};


// 실행 흐름:
// 1. allocate() 호출
// 2. cache_list.empty() 확인
// - 비어있으면: std::aligned_alloc() 호출
// - 있으면: cache_list.pop() 호출 (빠름!)
// 3. VM 실행
// 4. unique_ptr 소멸자 호출
// 5. free_cached() 호출
// 6. cache_list.size() < max_slots 확인
// - 작으면: cache_list.push() (다음 실행을 위해 보관)
// - 크면: std::free() 호출
```

10. 참고 자료

10.1 관련 문서

- **01-아키텍처-개요.md**: VM 컴포넌트의 전체 구조
- **02-VM-네이티브-컴파일.md**: 컴파일러와 런타임의 통합
- **03-병렬-실행-스케줄링.md**: 병렬 실행 시 메모리 관리

10.2 외부 라이브러리

ethash:

- Keccak256 최적화 구현
- https://github.com/chfast/ethash

BLAKE3:

- 고성능 해시 함수
- https://github.com/BLAKE3-team/BLAKE3

intx:

- 256비트 정수 참조 구현
- https://github.com/chfast/intx

10.3 CPU 명령어 세트

Intel Intrinsics Guide:

- https://www.intel.com/content/www/us/en/docs/intrinsics-guide/

x86-64-v3 ISA:

- AVX2, BMI2, FMA 등 포함
- https://en.wikipedia.org/wiki/X86-64#Microarchitecture_levels

10.4 최적화 기법

Compiler Intrinsics:

- `__builtin_addcll` : 캐리를 포함한 덧셈
- `__builtin_subcll` : 보로우를 포함한 뺄셈

Assembly Optimization:

- MULX: 고성능 곱셈
- SHLD/SHRD: 더블 시프트
- ADC/SBB: 캐리/보로우 전파

SIMD Optimization:

- AVX2: 256비트 벡터 연산
- AVX-512: 512비트 벡터 연산
- 마스크 연산 및 gather/scatter

결론

Monad VM의 런타임 최적화는 다음과 같은 핵심 원칙을 따릅니다:

1. **핫 패스 집중**: 가장 빈번한 연산에 최적화 집중
2. **하드웨어 활용**: CPU 명령어 세트를 최대한 활용
3. **메모리 효율**: 할당 오버헤드 최소화 및 캐시 친화적 설계
4. **컴파일 타임 최적화**: 가능한 모든 계산을 컴파일 타임에 수행
5. **분기 제거**: 예측 가능한 코드 경로 생성

이러한 최적화들이 결합되어 Monad는 기존 EVM 구현 대비 10배 이상의 성능 향상을 달성합니다.